```
!pip install torch pandas numpy scikit-learn
!pip install ta
import os
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
import ta
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Collecting ta
  Downloading ta-0.11.0.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from ta) (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from ta) (2.2.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->ta) (2.9.0.pc
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->ta) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->ta) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->ta)
Building wheels for collected packages: ta
  Building wheel for ta (setup.py) ... done
  Created wheel for ta: filename=ta-0.11.0-py3-none-any.whl size=29412 sha256=c33057c17efdac89f930a8cbbc0024ea86d670719a976f
  Stored in directory: /root/.cache/pip/wheels/5c/a1/5f/c6b85a7d9452057be4ce68a8e45d77ba34234a6d46581777c6
Successfully built ta
Installing collected packages: ta
Successfully installed ta-0.11.0
```

```
# Install darts library, which includes the Temporal Fusion Transformer model.
# We'll install with the 'torch' backend for deep learning models.
!pip install -q darts[torch] tensorflow

import pandas as pd
import numpy as np

# Import the necessary components from darts
from darts import TimeSeries
from darts.models import TFTModel
from darts.dataprocessing.transformers import Scaler
from darts.utils.likelihood_models import QuantileRegression

print("Darts library and TFTModel components imported successfully.")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 64.9/64.9 kB 1.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.3/46.3 kB 1.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 572.6/572.6 MB 878.2 kB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 72.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.6/16.6 MB 61.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 324.3/324.3 kB 18.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 219.8/219.8 kB 15.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 825.4/825.4 kB 37.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.5/87.5 kB 5.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 760.5/760.5 kB 33.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.8/3.8 MB 68.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 8.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 983.4/983.4 kB 35.2 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-cloud-aiplatform 1.143.0 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but
google-cloud-discoveryengine 0.13.12 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2,
tf-keras 2.19.0 requires tensorflow<2.20,>=2.19, but you have tensorflow 2.21.0 which is incompatible.
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 7.34.1 which is incompatible.
google-cloud-pubsub 2.36.0 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but you h
google-cloud-firestore 2.26.0 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but yo
google-cloud-bigtable 2.35.0 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but you
```

```
google-api-core 2.30.1 requires protobuf<7.0.0,>=4.25.8, but you have protobuf 7.34.1 which is incompatible.
opentelemetry-proto 1.38.0 requires protobuf<7.0,>=5.0, but you have protobuf 7.34.1 which is incompatible.
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.
grpc-google-iam-v1 0.14.3 requires protobuf!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but you have protob
wandb 0.25.1 requires protobuf!=5.28.0,!=5.29.0,<7,>4.21.0, but you have protobuf 7.34.1 which is incompatible.
ydf 0.15.0 requires protobuf<7.0.0,>=5.29.1, but you have protobuf 7.34.1 which is incompatible.
google-cloud-spanner 3.63.0 requires protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.20.2, but you
tensorflow-decision-forests 1.12.0 requires tensorflow==2.19.0, but you have tensorflow 2.21.0 which is incompatible.
tensorflow-text 2.19.0 requires tensorflow<2.20,>=2.19.0, but you have tensorflow 2.21.0 which is incompatible.
WARNING:darts.models:The StatsForecast module could not be imported. To enable support for the AutoARIMA, AutoETS and Crostc
Darts library and TFTModel components imported successfully.
```

```
import yfinance as yf
df= yf.download('RELIANCE.NS', start='2011-01-01', end='2024-01-01')
```

```
/tmp/ipykernel_3246/1610097061.py:2: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df= yf.download('RELIANCE.NS', start='2011-01-01', end='2024-01-01')
[*********************100%***********************]  1 of 1 completed
```

```
def add_technical_indicators(df):
    # RSI
    df['rsi'] = ta.momentum.rsi(df['Close'], window=14)
    # Bollinger Bands
    bollinger = ta.volatility.BollingerBands(close=df['Close'], window=20, window_dev=2)
    df['bb_high'] = bollinger.bollinger_hband()
    df['bb_low'] = bollinger.bollinger_lband()
    # Moving Average Slope
    df['ma_20'] = df['Close'].rolling(window=20).mean()
    df['ma_20_slope'] = df['ma_20'].diff()

    # Fill any NaNs from indicator calculations
    df.bfill(inplace=True)
    df.ffill(inplace=True)
    return df
```

```
from sklearn.preprocessing import MinMaxScaler

def select_and_scale_features(df, feature_cols=None):
    if feature_cols is None:
        # default feature set: O,H,L,C and a few indicators
        feature_cols = ['Open', 'High', 'Low', 'Close',
                        'rsi', 'bb_high', 'bb_low', 'ma_20', 'ma_20_slope']

    data = df[feature_cols].values  # shape: (num_samples, num_features)
    scaler = MinMaxScaler()
    data_scaled = scaler.fit_transform(data)
    return data_scaled, scaler, feature_cols
```

```
from torch.utils.data import Dataset, DataLoader

class ForexDataset(Dataset):
    def __init__(self, data, seq_length=80, prediction_length=5, feature_dim=8, target_column_idx=3):
        """
        data: numpy array of shape [num_samples, num_features]
        seq_length: how many timesteps in the input sequence
        prediction_length: how many future steps we want to predict
        feature_dim: total number of features in data (for dimension checking)
        target_column_idx: which column to use as the target (e.g., close=3)
        """
        self.data = data
        self.seq_length = seq_length
        self.pred_length = prediction_length
        self.feature_dim = feature_dim
        self.target_column_idx = target_column_idx

    def __len__(self):
        # The maximum starting index is total_length - seq_length - prediction_length
        return len(self.data) - self.seq_length - self.pred_length + 1

    def __getitem__(self, idx):
        # Input sequence
        x = self.data[idx : idx + self.seq_length]
        # Future price(s)
        y = self.data[idx + self.seq_length : idx + self.seq_length + self.pred_length, self.target_column_idx]
        return torch.tensor(x, dtype=torch.float32), torch.tensor(y, dtype=torch.float32)
```

```
import torch.nn as nn

class TimeSeriesTransformer(nn.Module):
    def __init__(
```

```
            self,
            feature_size=9,
            num_layers=2,
            d_model=64,
            nhead=8,
            dim_feedforward=256,
            dropout=0.1,
            seq_length=30,
            prediction_length=1
        ):
            super(TimeSeriesTransformer, self).__init__()

            # We'll embed each feature vector (feature_size) into a d_model-sized vector
            self.input_fc = nn.Linear(feature_size, d_model)

            # Positional Encoding (simple learnable or sinusoidal). We'll do a learnable here:
            self.pos_embedding = nn.Parameter(torch.zeros(1, seq_length, d_model))

           # Transformer Encoder
            encoder_layer = nn.TransformerEncoderLayer(
                d_model=d_model,
                nhead=nhead,
                dim_feedforward=dim_feedforward,
                dropout=dropout,
                activation="relu"
            )
            self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

            # Final output: we want to forecast `prediction_length` steps for 1 dimension (Close price).
            # If you want multi-step and multi-dimensional, adjust accordingly.
            self.fc_out = nn.Linear(d_model, prediction_length)

        def forward(self, src):
            """
            src shape: [batch_size, seq_length, feature_size]
            """
            batch_size, seq_len, _ = src.shape

            # First project features into d_model
            src = self.input_fc(src)  # -> [batch_size, seq_length, d_model]

            # Add positional embedding
            # pos_embedding -> [1, seq_length, d_model], so broadcast along batch dimension
            src = src + self.pos_embedding[:, :seq_len, :]

            # Transformer expects shape: [sequence_length, batch_size, d_model]
            src = src.permute(1, 0, 2)  # -> [seq_length, batch_size, d_model]

            # Pass through the transformer
            encoded = self.transformer_encoder(src)  # [seq_length, batch_size, d_model]

            # We only want the output at the last time step for forecasting the future
            last_step = encoded[-1, :, :]  # [batch_size, d_model]

            out = self.fc_out(last_step)  # [batch_size, prediction_length]
            return out
```

```
def train_transformer_model(
    model,
    train_loader,
    val_loader=None,
    lr=1e-3,
    epochs=30,
    device='cpu'
):
    criterion = nn.MSELoss()  # For regression on price
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    model.to(device)

    for epoch in range(epochs):
        model.train()
        train_losses = []
        for x_batch, y_batch in train_loader:
            x_batch = x_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            output = model(x_batch)  # output shape: [batch_size, prediction_length]
            loss = criterion(output, y_batch)
            loss.backward()
            optimizer.step()
```

```
            train_losses.append(loss.item())

        mean_train_loss = np.mean(train_losses)

        if val_loader is not None:
            model.eval()
            val_losses = []
            with torch.no_grad():
                for x_val, y_val in val_loader:
                    x_val = x_val.to(device)
                    y_val = y_val.to(device)
                    output_val = model(x_val)
                    loss_val = criterion(output_val, y_val)
                    val_losses.append(loss_val.item())
            mean_val_loss = np.mean(val_losses)
            print(f"Epoch [{epoch+1}/{epochs}], Train Loss: {mean_train_loss:.6f}, Val Loss: {mean_val_loss:.6f}")
        else:
            print(f"Epoch [{epoch+1}/{epochs}], Train Loss: {mean_train_loss:.6f}")

    return model
```

```
import matplotlib.pyplot as plt
import numpy as np
import torch

def evaluate_model(model, test_loader, scaler, feature_cols, target_col_idx,
                   window_width=10, start_index=0, pred_length=1, device='cpu'):
    """
    Evaluates the model on test data and compares predictions with actual prices.
    Plots real vs. predicted values within a given window width and starting index.

    Parameters:
        model: Trained PyTorch model.
        test_loader: DataLoader for test data.
        scaler: MinMaxScaler (used to inverse transform predictions and real values).
        feature_cols: List of feature column names.
        target_col_idx: Index of the "Close" price in feature columns.
        window_width: Number of points to plot for real vs. predicted prices.
        start_index: The index in the test dataset from which to start plotting.
        pred_length: Number of future values predicted by the model.
        device: 'cpu' or 'cuda' for model inference.
    """
    model.eval()
    real_prices = []
    predicted_prices = []

    with torch.no_grad():
        for x_batch, y_batch in test_loader:
            x_batch = x_batch.to(device)

            # Get model predictions
            predictions = model(x_batch).cpu().numpy()  # shape: [batch_size, pred_length]
            y_batch = y_batch.cpu().numpy()  # shape: [batch_size, pred_length]

            for i in range(len(predictions)):
                # Create dummy inputs for inverse scaling
                dummy_pred = np.zeros((pred_length, len(feature_cols)))
                dummy_pred[:, target_col_idx] = predictions[i]  # Assign predicted future prices

                dummy_real = np.zeros((pred_length, len(feature_cols)))
                dummy_real[:, target_col_idx] = y_batch[i]  # Assign real future prices

                # Inverse transform both predicted and actual prices
                pred_inversed = scaler.inverse_transform(dummy_pred)[:, target_col_idx]
                real_inversed = scaler.inverse_transform(dummy_real)[:, target_col_idx]

                # Store values
                predicted_prices.extend(pred_inversed)
                real_prices.extend(real_inversed)

    # Convert lists to numpy arrays
    real_prices = np.array(real_prices).flatten()
    predicted_prices = np.array(predicted_prices).flatten()

    # -------------------------
    # Compute Accuracy Metrics
    # -------------------------
    mse = np.mean((real_prices - predicted_prices) ** 2)
    mae = np.mean(np.abs(real_prices - predicted_prices))

    print(f"Model Evaluation:\n  - Mean Squared Error (MSE): {mse:.4f}")
    print(f"  - Mean Absolute Error (MAE): {mae:.4f}")
```

```
    # -------------------------
    # Adjust Start Index and Window Width for Plot
    # -------------------------
    if start_index < 0 or start_index >= len(real_prices):
        print(f"Warning: start_index {start_index} is out of bounds. Using 0 instead.")
        start_index = 0

    end_index = min(start_index + window_width * pred_length, len(real_prices))  # Adjust for multi-step forecasts

    # -------------------------
    # Plot Real vs. Predicted Prices
    # -------------------------
    plt.figure(figsize=(12, 6))
    plt.plot(range(start_index, end_index), real_prices[start_index:end_index],
             label="Real Close Prices", linestyle="dashed", marker='o')
    plt.plot(range(start_index, end_index), predicted_prices[start_index:end_index],
             label="Predicted Close Prices", linestyle="-", marker='x')
    plt.title(f"Real vs. Predicted Close Prices (From index {start_index}, {window_width} Windows, {pred_length} Steps Each
    plt.xlabel("Time Steps")
    plt.ylabel("Close Price")
    plt.legend()
    plt.show()
```

```
import ta
import yfinance as yf
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt

# Redefine ForexDataset for binary classification
class ForexDataset(Dataset):
    def __init__(self, data, seq_length=80, prediction_length=5, feature_dim=8, close_col_idx=3):
        """
        data: numpy array of shape [num_samples, num_features]
        seq_length: how many timesteps in the input sequence (for X)
        prediction_length: how many future steps to look ahead for the target (e.g., 5 for t+5)
        feature_dim: total number of features in data
        close_col_idx: index of the 'Close' price column in the features
        """
        self.data = data
        self.seq_length = seq_length
        self.pred_length = prediction_length
        self.feature_dim = feature_dim
        self.close_col_idx = close_col_idx

    def __len__(self):
        # We need enough data for the sequence, plus the pred_length days into the future for the target
        # So, the last possible 'idx' must allow for seq_length + pred_length-1 elements
        return len(self.data) - self.seq_length - self.pred_length + 1

    def __getitem__(self, idx):
        # Input sequence (t-seq_length to t-1)
        x = self.data[idx : idx + self.seq_length]

        # Current closing price at the end of the input sequence (time t-1)
        # This is `t` price for the comparison
        current_close_price = self.data[idx + self.seq_length - 1, self.close_col_idx]

        # Future closing price at time t + pred_length - 1 (the 't+5' price if pred_length is 5)
        # This is `t+5` price for the comparison
        future_close_price = self.data[idx + self.seq_length + self.pred_length - 1, self.close_col_idx]

        # Calculate binary target: 1.0 if future price > current price, 0.0 otherwise
        y = 1.0 if future_close_price > current_close_price else 0.0

        # Return y as a single-element tensor, as expected by BCEWithLogitsLoss
        return torch.tensor(x, dtype=torch.float32), torch.tensor([y], dtype=torch.float32)

# Redefine TimeSeriesTransformer for binary classification output
class TimeSeriesTransformer(nn.Module):
    def __init__(
        self,
        feature_size=9,
        num_layers=2,
        d_model=64,
```

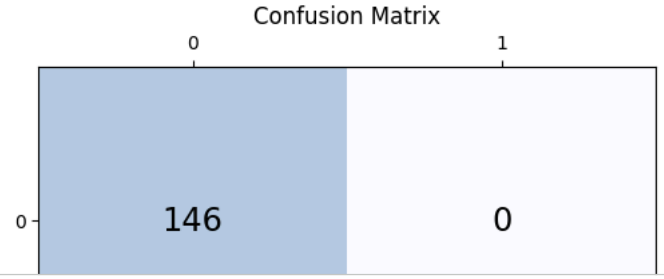
```
            nhead=8,
            dim_feedforward=256,
            dropout=0.1,
            seq_length=30,
            output_dim=1 # Changed from prediction_length to output_dim, now 1 for binary classification
        ):
            super(TimeSeriesTransformer, self).__init__()

            self.input_fc = nn.Linear(feature_size, d_model)
            self.pos_embedding = nn.Parameter(torch.zeros(1, seq_length, d_model))

            encoder_layer = nn.TransformerEncoderLayer(
                d_model=d_model,
                nhead=nhead,
                dim_feedforward=dim_feedforward,
                dropout=dropout,
                activation="relu"
            )
            self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

            # Output layer now produces a single logit for binary classification
            self.fc_out = nn.Linear(d_model, output_dim)

        def forward(self, src):
            batch_size, seq_len, _ = src.shape
            src = self.input_fc(src)
            src = src + self.pos_embedding[:, :seq_len, :]
            src = src.permute(1, 0, 2)
            encoded = self.transformer_encoder(src)
            last_step = encoded[-1, :, :]
            out = self.fc_out(last_step) # Output logits, BCEWithLogitsLoss will apply sigmoid
            return out

    # Redefine train_transformer_model for binary classification
    def train_transformer_model(
        model,
        train_loader,
        val_loader=None,
        lr=1e-3,
        epochs=30,
        device='cpu'
    ):
        criterion = nn.BCEWithLogitsLoss()  # For binary classification (combines Sigmoid and BCE)
        optimizer = torch.optim.Adam(model.parameters(), lr=lr)

        model.to(device)

        for epoch in range(epochs):
            model.train()
            train_losses = []
            for x_batch, y_batch in train_loader:
                x_batch = x_batch.to(device)
                y_batch = y_batch.to(device) # y_batch is already [batch_size, 1]

                optimizer.zero_grad()
                output = model(x_batch)  # output shape: [batch_size, 1] (logits)
                loss = criterion(output, y_batch)
                loss.backward()
                optimizer.step()
                train_losses.append(loss.item())

            mean_train_loss = np.mean(train_losses)

            if val_loader is not None:
                model.eval()
                val_losses = []
                with torch.no_grad():
                    for x_val, y_val in val_loader:
                        x_val = x_val.to(device)
                        y_val = y_val.to(device)
                        output_val = model(x_val)
                        loss_val = criterion(output_val, y_val)
                        val_losses.append(loss_val.item())
                mean_val_loss = np.mean(val_losses)
                print(f"Epoch [{epoch+1}/{epochs}], Train Loss: {mean_train_loss:.6f}, Val Loss: {mean_val_loss:.6f}")
            else:
                print(f"Epoch [{epoch+1}/{epochs}], Train Loss: {mean_train_loss:.6f}")

        return model

    # New evaluation function for binary classification
    def evaluate_classification_model(model, test_loader, device='cpu'):
```

```
    """
    Evaluates the model on test data for binary classification.
    Prints classification metrics and a confusion matrix.
    """
    model.eval()
    all_true_labels = []
    all_predicted_labels = []

    with torch.no_grad():
        for x_batch, y_batch in test_loader:
            x_batch = x_batch.to(device)
            y_batch = y_batch.cpu().numpy()  # True labels (0 or 1)

            # Get model logits
            logits = model(x_batch).cpu().numpy() # shape: [batch_size, 1]

            # Apply sigmoid and threshold (0.5) for binary prediction
            probabilities = 1 / (1 + np.exp(-logits)) # Sigmoid function
            predictions = (probabilities >= 0.5).astype(int) # Binary predictions

            all_true_labels.extend(y_batch.flatten())
            all_predicted_labels.extend(predictions.flatten())

    all_true_labels = np.array(all_true_labels)
    all_predicted_labels = np.array(all_predicted_labels)

    # Compute Classification Metrics
    accuracy = accuracy_score(all_true_labels, all_predicted_labels)
    precision = precision_score(all_true_labels, all_predicted_labels, zero_division=0)
    recall = recall_score(all_true_labels, all_predicted_labels, zero_division=0)
    f1 = f1_score(all_true_labels, all_predicted_labels, zero_division=0)
    conf_matrix = confusion_matrix(all_true_labels, all_predicted_labels)

    print(f"Model Classification Evaluation:")
    print(f"  - Accuracy: {accuracy:.4f}")
    print(f"  - Precision: {precision:.4f}")
    print(f"  - Recall: {recall:.4f}")
    print(f"  - F1-Score: {f1:.4f}")
    print(f"  - Confusion Matrix:\n{conf_matrix}")

    # Plot Confusion Matrix
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.matshow(conf_matrix, cmap=plt.cm.Blues, alpha=0.3)
    for i in range(conf_matrix.shape[0]):
        for j in range(conf_matrix.shape[1]):
            ax.text(x=j, y=i,s=conf_matrix[i, j], va='center', ha='center', size='xx-large')
    plt.xlabel('Predicted labels')
    plt.ylabel('True labels')
    plt.title('Confusion Matrix')
    plt.show()

    return accuracy, precision, recall, f1, conf_matrix

# --- Main Script Execution (modified for classification) ---

# Original code for data fetching and preprocessing
df= yf.download('RELIANCE.NS', start='2011-01-01', end='2024-01-01')

# Flatten MultiIndex columns if they exist.
if isinstance(df.columns, pd.MultiIndex):
    new_columns = []
    for col_level0, col_level1 in df.columns:
        if col_level0 == 'Price':
            new_columns.append(col_level1)
        elif col_level0 == 'Volume':
            new_columns.append(col_level0)
        else:
            new_columns.append(f"{col_level0}_{col_level1}")
    df.columns = new_columns

df.rename(columns={
    'Close_RELIANCE.NS': 'Close',
    'High_RELIANCE.NS': 'High',
    'Low_RELIANCE.NS': 'Low',
    'Open_RELIANCE.NS': 'Open'
}, inplace=True)

df = add_technical_indicators(df)

# Select features and scale
data_scaled, scaler, feature_cols = select_and_scale_features(df)
close_col_idx = feature_cols.index('Close') # Use close_col_idx now
```

```
# -------------------------
# 2. Create Dataset & Dataloaders
# -------------------------
seq_length = 30
pred_length = 5  # Still 5 for T+5 day target calculation in dataset

dataset = ForexDataset(data_scaled, seq_length, pred_length, len(feature_cols), close_col_idx)

# Train/Validation/Test Split (80% train, 10% val, 10% test)
train_size = int(len(dataset) * 0.8)
val_size = int(len(dataset) * 0.1)
test_size = len(dataset) - train_size - val_size

# Perform sequential splitting
train_dataset = torch.utils.data.Subset(dataset, range(0, train_size))
val_dataset = torch.utils.data.Subset(dataset, range(train_size, train_size + val_size))
test_dataset = torch.utils.data.Subset(dataset, range(train_size + val_size, len(dataset)))

batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# -------------------------
# 3. Create and Train Transformer Model
# -------------------------
model = TimeSeriesTransformer(
    feature_size=len(feature_cols),
    num_layers=2,
    d_model=64,
    nhead=8,
    dim_feedforward=256,
    dropout=0.1,
    seq_length=seq_length,
    output_dim=1 # Now 1 for binary classification
)

device = 'cuda' if torch.cuda.is_available() else 'cpu'
trained_model = train_transformer_model(model, train_loader, val_loader, lr=1e-3, epochs=30, device=device)

# -------------------------
# 4. Evaluate the Classification Model
# -------------------------
print("\n--- Model Evaluation on Test Set ---")
evaluate_classification_model(trained_model, test_loader, device=device)


# -------------------------
# 5. Out-of-Sample Forecast
# -------------------------
print("\n--- Out-of-Sample Forecast ---")

# Get the last `seq_length` data points from the preprocessed df for forecasting
input_df_for_forecast = df.iloc[-seq_length:]

# Select and scale features for this input sequence
input_sequence_scaled, _, _ = select_and_scale_features(input_df_for_forecast, feature_cols=feature_cols)

# Ensure the input sequence has the correct shape [1, seq_length, num_features]
input_tensor = torch.tensor(input_sequence_scaled, dtype=torch.float32).unsqueeze(0).to(device)

trained_model.eval() # Set model to evaluation mode
with torch.no_grad(): # Disable gradient calculations
    logits_forecast = trained_model(input_tensor).cpu().numpy().flatten()
    # Apply sigmoid and threshold (0.5) for binary prediction
    probability_forecast = 1 / (1 + np.exp(-logits_forecast)) # Sigmoid function
    binary_prediction = (probability_forecast >= 0.5).astype(int)

print(f"\nModel's predicted T+{pred_length} day movement (1=Up, 0=Down or Flat): {binary_prediction[0]}")

# 6. Fetch actual future data and compare
last_date_of_trained_data = df.index[-1]
start_actual_fetch_date = last_date_of_trained_data + pd.Timedelta(days=1)
end_actual_fetch_date = start_actual_fetch_date + pd.Timedelta(days=15) # Fetch for a longer period to ensure pred_length t

print(f"\nAttempting to fetch actual data from yfinance from {start_actual_fetch_date.date()} to {end_actual_fetch_date.dat
actual_future_df_raw = yf.download('RELIANCE.NS', start=start_actual_fetch_date, end=end_actual_fetch_date)

actual_future_close_prices_raw = actual_future_df_raw['Close']

if actual_future_close_prices_raw.empty:
```

```
    print(f"No actual data found for RELIANCE.NS from {start_actual_fetch_date.date()} to {end_actual_fetch_date.date()}. (
else:
    # We need the current close price (last price from the training data)
    last_known_close_price = df['Close'].iloc[-1]

    # Filter for trading days and get the (pred_length)-th day
    actual_future_trading_days = actual_future_close_prices_raw.dropna()

    if len(actual_future_trading_days) >= pred_length:
        # Get the close price for the actual T+pred_length day
        t_plus_5_actual_close_price = actual_future_trading_days.iloc[pred_length-1] # pred_length-1 for 0-indexed
        actual_binary_outcome = 1 if t_plus_5_actual_close_price > last_known_close_price else 0

        print(f"\nLast known close price (before forecast window): {last_known_close_price:.2f}")
        print(f"Actual T+{pred_length} day close price: {t_plus_5_actual_close_price:.2f}")
        print(f"Actual T+{pred_length} day movement (1=Up, 0=Down or Flat): {actual_binary_outcome}")
        print(f"Predicted T+{pred_length} day movement: {binary_prediction[0]}")

        if binary_prediction[0] == actual_binary_outcome:
            print("\nForecast matches actual outcome!")
        else:
            print("\nForecast DOES NOT match actual outcome.")

        # Plotting for binary outcomes
        fig, ax = plt.subplots(figsize=(6, 4))
        labels = ['Predicted', 'Actual']
        outcomes = [binary_prediction[0], actual_binary_outcome]
        colors = ['blue' if o == 1 else 'red' for o in outcomes]

        ax.bar(labels, outcomes, color=colors, width=0.4)
        ax.set_ylabel('Outcome (1=Up, 0=Down/Flat)')
        ax.set_title(f'Predicted vs. Actual T+{pred_length} Day Movement')
        ax.set_yticks([0, 1]) # Ensure y-axis shows 0 and 1 clearly
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.show()

    else:
        print(f"Not enough actual future trading days ({len(actual_future_trading_days)}) to determine T+{pred_length} out(
```

```
/tmp/ipykernel_3246/503595390.py:196: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df= yf.download('RELIANCE.NS', start='2011-01-01', end='2024-01-01')
[*********************100%***********************]  1 of 1 completed
/tmp/ipykernel_3246/503595390.py:76: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False becaus
  self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
Epoch [1/30], Train Loss: 0.698925, Val Loss: 0.695241
Epoch [2/30], Train Loss: 0.692919, Val Loss: 0.689314
Epoch [3/30], Train Loss: 0.689861, Val Loss: 0.686940
Epoch [4/30], Train Loss: 0.688427, Val Loss: 0.707696
Epoch [5/30], Train Loss: 0.689304, Val Loss: 0.692644
Epoch [6/30], Train Loss: 0.683258, Val Loss: 0.693960
Epoch [7/30], Train Loss: 0.683196, Val Loss: 0.694017
Epoch [8/30], Train Loss: 0.684901, Val Loss: 0.693895
Epoch [9/30], Train Loss: 0.682184, Val Loss: 0.693046
Epoch [10/30], Train Loss: 0.684145, Val Loss: 0.695611
Epoch [11/30], Train Loss: 0.679547, Val Loss: 0.702429
Epoch [12/30], Train Loss: 0.681627, Val Loss: 0.710639
Epoch [13/30], Train Loss: 0.677125, Val Loss: 0.701173
Epoch [14/30], Train Loss: 0.677047, Val Loss: 0.702087
Epoch [15/30], Train Loss: 0.679430, Val Loss: 0.703202
Epoch [16/30], Train Loss: 0.673822, Val Loss: 0.697884
Epoch [17/30], Train Loss: 0.678232, Val Loss: 0.704224
Epoch [18/30], Train Loss: 0.681070, Val Loss: 0.690235
Epoch [19/30], Train Loss: 0.672253, Val Loss: 0.697109
Epoch [20/30], Train Loss: 0.673233, Val Loss: 0.715460
Epoch [21/30], Train Loss: 0.673245, Val Loss: 0.717727
Epoch [22/30], Train Loss: 0.673674, Val Loss: 0.709226
Epoch [23/30], Train Loss: 0.672916, Val Loss: 0.694662
Epoch [24/30], Train Loss: 0.673217, Val Loss: 0.712505
Epoch [25/30], Train Loss: 0.673114, Val Loss: 0.705772
Epoch [26/30], Train Loss: 0.666990, Val Loss: 0.690228
Epoch [27/30], Train Loss: 0.669342, Val Loss: 0.704804
Epoch [28/30], Train Loss: 0.666089, Val Loss: 0.716667
Epoch [29/30], Train Loss: 0.669941, Val Loss: 0.714843
Epoch [30/30], Train Loss: 0.662922, Val Loss: 0.703804

--- Model Evaluation on Test Set ---
Model Classification Evaluation:
  - Accuracy: 0.4669
  - Precision: 1.0000
  - Recall: 0.0117
  - F1-Score: 0.0231
  - Confusion Matrix:
[[146   0]
 [169   2]]
```



```
import collections
t_plus_5_actual_close_price = actual_future_trading_days.iloc[pred_length-1] # pred_length-1 for 0-
evaluate_model(trained_model, test_loader, scaler, feature_cols, target_col_idx, pred_length=pred_length, device=device)

import yfinance as yf
import pandas as pd
import numpy as np
import torch
import matplotlib.pyplot as plt

# Assume df, trained_model, scaler, feature_cols, target_col_idx, seq_length, pred_length, device are available from previ

# 1. Get the last `seq_length` data points from the preprocessed df
# These will serve as the input to forecast the next `pred_length` days.
# It's important to use the df *after* technical indicators have been added.
input_df_for_forecast = df.iloc[-seq_length:]

# 2. Select and scale features for this input sequence
# We use the same `select_and_scale_features` function and the `scaler` fitted on the training data.
input_sequence_scaled, _, _ = select_and_scale_features(input_df_for_forecast, feature_cols=feature_cols)

# Ensure the input sequence has the correct shape [1, seq_length, num_features]
input_tensor = torch.tensor(input_sequence_scaled, dtype=torch.float32).unsqueeze(0).to(device)

# 3. Make prediction
trained_model.eval() # Set model to evaluation mode
with torch.no_grad(): # Disable gradient calculations
    predicted_scaled_values = trained_model(input_tensor).cpu().numpy().flatten() # shape: [pred_length]

# 4. Inverse transform predictions
# To inverse_transform, we need an array with the same number of features as the original scaled data.
# We fill the target column (Close price) with predictions and other columns with zeros (they will be ignored by scaler for
dummy_array_for_inverse = np.zeros((pred_length, len(feature_cols)))
dummy_array_for_inverse[:, target_col_idx] = predicted_scaled_values
predicted_prices_inversed = scaler.inverse_transform(dummy_array_for_inverse)[:, target_col_idx]

print(f"\nModel's predicted {pred_length}-day Close prices (starting after {df.index[-1].date()}):")
print(predicted_prices_inversed)

# 5. Fetch actual future data from yfinance
# The original df ended on 2024-01-01 (as per yf.download end parameter), but the last actual data point in processed df is
# So, we need actual data from 2023-12-30 onwards to compare with our forecast.
last_date_of_trained_data = df.index[-1]
start_actual_fetch_date = last_date_of_trained_data + pd.Timedelta(days=1) # Start one day after the last date in our proce
# Fetch enough data to cover the pred_length trading days
# We'll fetch for a longer period to ensure we get 5 trading days
end_actual_fetch_date = start_actual_fetch_date + pd.Timedelta(days=15) # Example: fetch 15 calendar days to get ~5-10 trad

print(f"\nAttempting to fetch actual data from yfinance from {start_actual_fetch_date.date()} to {end_actual_fetch_date.dat
actual_future_df_raw = yf.download('RELIANCE.NS', start=start_actual_fetch_date, end=end_actual_fetch_date)

# 6. Prepare actual data - just need the Close prices for comparison
actual_future_close_prices_raw = actual_future_df_raw['Close']

if actual_future_close_prices_raw.empty:
    print(f"No actual data found for RELIANCE.NS from {start_actual_fetch_date.date()} to {end_actual_fetch_date.date()}. (
else:
    # Ensure we only compare for the `pred_length` days
    actual_future_close_prices_for_comparison = actual_future_close_prices_raw.head(pred_length).values

    if len(actual_future_close_prices_for_comparison) < pred_length:
        print(f"Warning: Only {len(actual_future_close_prices_for_comparison)} actual future close prices available from yf
        # Adjust comparison length to available actuals
        comparison_length = len(actual_future_close_prices_for_comparison)
        predicted_prices_for_comparison = predicted_prices_inversed[:comparison_length]
    else:
        comparison_length = pred_length
        predicted_prices_for_comparison = predicted_prices_inversed
```